

Evidencia 2. Presentación FINAL del reto

Equipo 6:

Fernando Bustos Monsiváis - A00829931

Rodolfo Charles Wah - A01383393

Lautaro Gabriel Coteja - A01571214

Carlos Rogelio Villarreal Treviño - A00832489

Marcelo Hernández Almada - A01194283

Escuela de Ingeniería y Ciencias, Instituto Tecnológico y de Estudios Superiores de Monterrey

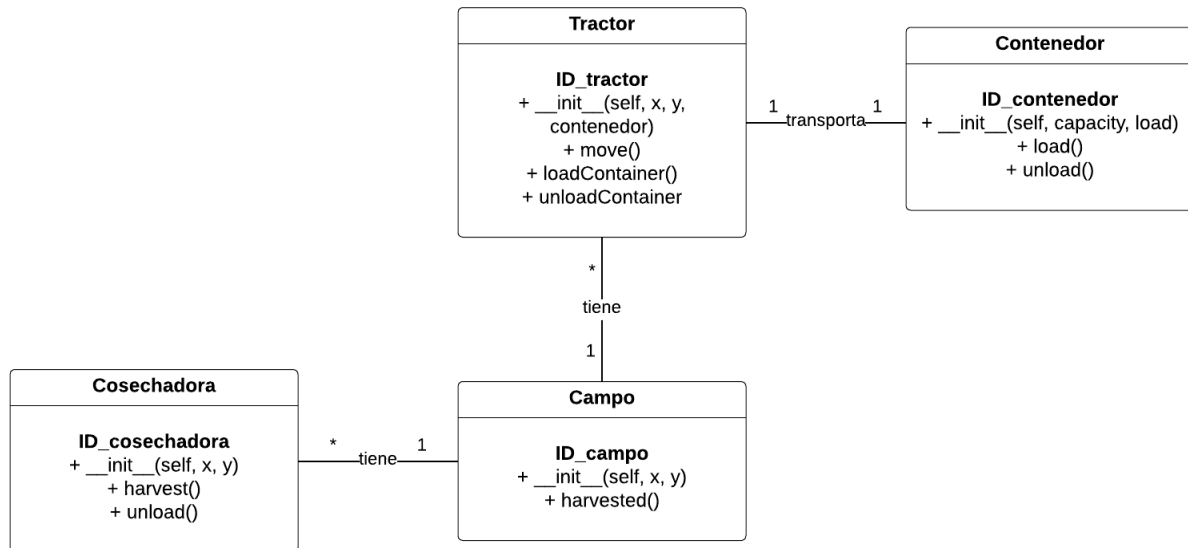
TC2008B.302: Modelación de sistemas multiagentes con gráficas computacionales

Maestros: Dr. Luis Alberto Muñoz Ubando y Dr. Raúl Valente Ramírez Velarde

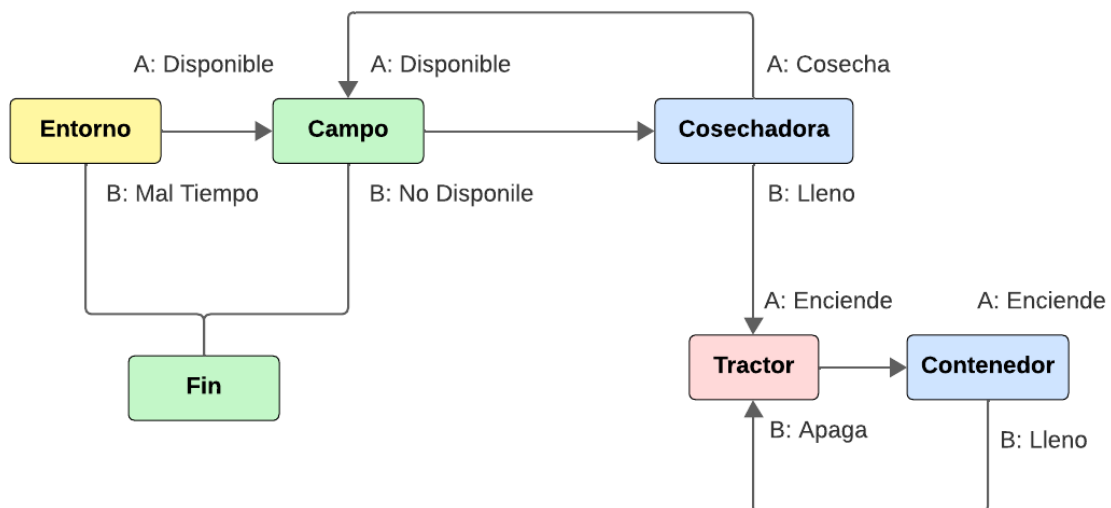
Viernes 01 de diciembre de 2023

1. Diagramas de clase y protocolos de interacción finales

1.1 Diagrama de clase presentando los distintos agentes involucrados



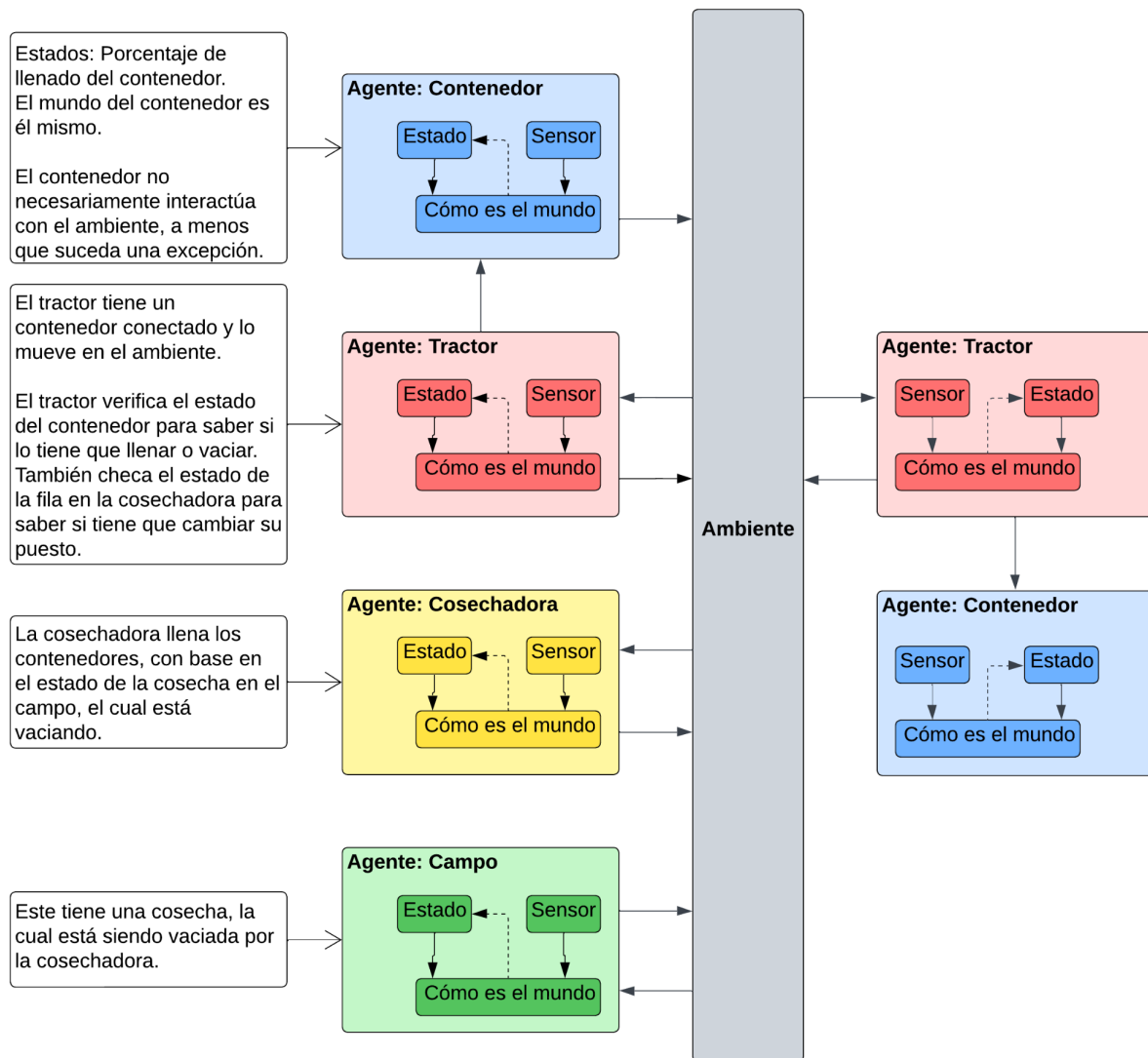
1.2 Diagrama de protocolos de interacción



1.3 Tabla de protocolos de interacción

Agente	Evento	Acciones
Cosechadora	Moveirse - move(self, x, y)	Se traslada a través del campo. Checa su entorno para verificar que no está siendo bloqueado por ningún obstáculo.
	Cosechar - harvest(self, x, y)	Agarra la siembra de un espacio de la matriz, llama la función de cosecha del campo con las coordenadas para que se marque este espacio como vacío. Llena el contenedor del tractor y llama su función de load.
Tractor	Moveirse - move(self, x, y)	Se traslada a través del campo. Checa su entorno para verificar que no está siendo bloqueado por ningún obstáculo
Campo	Cosechar - harvest(self, x, y)	Se vacía el campo en el cual se está cosechando.
Contenedor	Cargar - load(self, amount)	El contenedor es cargado con la siembra que es cosechada.

1.4 Modelación del sistema multiagentes necesario para simular el reto



2. Implementación completa de los agentes

Clase Container:

```
import agentpy as ap

class Container(ap.Agent):
    def __init__(self, field):
        self.field = field
        self.load = 0
        self.position = [1, 1]

    def loadContainer(self, amount):
        self.load += amount

    def move(self, newPosition):
        self.position = newPosition
```

Clase Field:

```
import random
import agentpy as ap

class Field(ap.Agent):
    def __init__(self, field, environmentRows, environmentColumns):
        self.field = field
        self.environmentRows = environmentRows
        self.environmentColumns = environmentColumns

    #define a function that determines if the specified location is a
    terminal state
    def is_terminal_state(self, current_row_index, current_column_index):
        #if the reward for this location is -1, then it is not a terminal
        state (i.e., it is a 'white square')
        if self.isEmpty(): return True

        if (self.field[current_row_index, current_column_index] == -1 or
            self.field[current_row_index, current_column_index] == 1): return False
```

```

        return True

    def getIndicesToChange(self, amountOfObstacles):
        indices_to_change = set()
        while len(indices_to_change) < amountOfObstacles:
            row = random.randint(0, self.environmentRows - 1)
            col = random.randint(0, self.environmentColumns - 1)
            indices_to_change.add((row, col))
        return indices_to_change

    def createObstacles(self, indices_to_change):
        # Change the values at the selected indices to -100
        for row, col in indices_to_change:
            self.field[row][col] = -100

        for i in range(len(self.field)):
            for j in range(len(self.field[i])):
                if i == 0 or i == len(self.field) - 1 or j == 0 or j ==
len(self.field[i]) - 1:
                    self.field[i][j] = -100
        return self.field

    def isEmpty(self):
        for row in self.field:
            for element in row:
                if element == 1: return False

        return True

```

Clase Harvester:

```

import agentpy as ap
import numpy as np

class Harvester(ap.Agent):
    def __init__(self, field, q_values):
        self.field = field
        self.position = [1, 1]
        self.rows = len(field)

```

```

self.cols = len(field[0])
self.actions = ['up', 'right', 'down', 'left']
self.q_values = q_values

#define an epsilon greedy algorithm that will choose which action to
take next (i.e., where to move next)
def get_next_action(self, current_row_index, current_column_index,
epsilon):
    #if a randomly chosen value between 0 and 1 is less than epsilon,
    #then choose the most promising value from the Q-table for this
state.
    if np.random.random() < epsilon:
        return np.argmax(self.q_values[current_row_index,
current_column_index])
    else: #choose a random action
        return np.random.randint(4)

#define a function that will get the next location based on the chosen
action
def get_next_location(self, current_row_index, current_column_index,
action_index):
    new_row_index = current_row_index
    new_column_index = current_column_index
    if self.actions[action_index] == 'up' and current_row_index > 0:
        new_row_index -= 1
    elif self.actions[action_index] == 'right' and current_column_index
< self.cols - 1:
        new_column_index += 1
    elif self.actions[action_index] == 'down' and current_row_index <
self.rows - 1:
        new_row_index += 1
    elif self.actions[action_index] == 'left' and current_column_index
> 0:
        new_column_index -= 1

    self.position = [new_row_index, new_column_index]

```

```

        return new_row_index, new_column_index

    def harvest(self):
        if self.field[self.position[0], self.position[1]] == 1:
            self.field[self.position[0], self.position[1]] = -1

```

Class Tractor:

```

import agentpy as ap
class Tractor(ap.Agent):
    def __init__(self, container, field):
        self.field = field
        self.container = container
        self.position = [1, 1]
        self.rows = len(field)
        self.cols = len(field[0])

    def move(self, newPosition):
        self.position = newPosition
        self.moveContainer(newPosition)

    def moveContainer(self, newPosition):
        self.container.move(newPosition)

    def loadContainer(self, amount):
        self.container.loadContainer(amount)

```

main:

```

import numpy as np
from Harvester import Harvester
from Tractor import Tractor
from Container import Container
from Field import Field
import json

environmentRows = 11
environmentColumns = 11
field = np.full((environmentRows, environmentColumns), 1)

```

```

q_values = np.zeros((environmentRows, environmentColumns, 4))

fieldAgent = Field(field, environmentRows, environmentColumns)
actions = ['up', 'right', 'down', 'left']

indicesToChange = fieldAgent.getIndicesToChange(4)
field = fieldAgent.createObstacles(indicesToChange)

for row in field:
    print(row)

#define training parameters
epsilon = 0.9 #the percentage of time when we should take the best action
(instead of a random action)
discount_factor = 0.9 #discount factor for future rewards
learning_rate = 0.9 #the rate at which the AI agent should learn

dic = {}
#run through 1000 training episodes

for episode in range(10000):
    #refresh field
    field = np.full((environmentRows, environmentColumns), 1)
    fieldAgent = Field(field, environmentRows, environmentColumns)
    field = fieldAgent.createObstacles(indicesToChange)

    #refresh agents
    container = Container(field)
    tractor = Tractor(container, field)
    harvesterAgent = Harvester(field, q_values)
    fieldAgent = Field(field, environmentRows, environmentColumns)

    #get the starting location for this episode
    row_index, column_index = 1, 1
    episodeDic = {}

```

```

#continue taking actions (i.e., moving) until we reach a terminal state
#(i.e., until we reach the item packaging area or crash into an item
storage location)
cont = 0
success = False
while not fieldAgent.is_terminal_state(row_index, column_index) :
    #choose which action to take (i.e., where to move next)
    action_index = harvesterAgent.get_next_action(row_index,
column_index, epsilon)

    #perform the chosen action, and transition to the next state (i.e.,
move to the next location)
    old_row_index, old_column_index = row_index, column_index #store
the old row and column indexes
    row_index, column_index =
harvesterAgent.get_next_location(row_index, column_index, action_index)
    tractor.move([row_index, column_index])

    #receive the reward for moving to the new state, and calculate the
temporal difference
    reward = field[row_index, column_index]

    if reward == 1:
        harvesterAgent.harvest()
        tractor.loadContainer(1)
    if fieldAgent.isEmpty():
        reward = 100
        success = True

    old_q_value = q_values[old_row_index, old_column_index,
action_index]
    temporal_difference = reward + (discount_factor *
np.max(q_values[row_index, column_index])) - old_q_value

    #update the Q-value for the previous state and action pair
    new_q_value = old_q_value + (learning_rate * temporal_difference)

```

```

        q_values[old_row_index, old_column_index, action_index] =
new_q_value
        cont += 1

        episodeDic[f"step{cont}"] = {
            "harvesterPosition": harvesterAgent.position,
            "tractorPosition": tractor.position,
            "containerPosition": tractor.container.position,
            "containerLoad": tractor.container.load,
            "field": [row[:] for row in field.tolist()],
        }

        dic[f"episode{episode}"] = {
            "steps": episodeDic,
            "success": success
        }
print('Training complete!')

with open("100reward1000episodesFieldEncapsulated.json", "w") as outfile:
    json.dump(dic, outfile)

```

Salida:

```

[-100 -100 -100 -100 -100 -100 -100 -100 -100 -100 -100]
[-100    1    1    1    1    1    1    1    1    1 -100]
[-100    1    1    1    1    1    1    1    1    1 -100]
[-100    1    1    1    1    1    1    1    1    1 -100]
[-100    1    1    1    1    1    1    1    1    1 -100]
[-100 -100    1    1    1    1    1    1    1    1 -100]
[-100    1    1    1    1    1    1    1    1    1 -100]
[-100    1    1    1    1    1    1    1    1    1 -100]
[-100    1    1    1    1    1    1    1    1    1 -100]
[-100    1    1    1    1    1    1    1    1    1 -100]
[-100    1    1 -100    1    1    1    1    1    1 -100]
[-100 -100 -100 -100 -100 -100 -100 -100 -100 -100 -100]
Training complete!

```

El modelo de aprendizaje se encuentra en main, utilizando métodos internos de cada clase, al final la cosechadora interactúa con el modelo de aprendizaje y tanto el tractor como el contenedor la siguen.

La cosechadora encuentra sus patrones de movimiento en torno al modelo de Q-Learning, va avanzando por el campo cosechando y se encarga de que este quede completamente vacío.

3. Implementación completa de la interfaz gráfica de la simulación

```
using UnityEngine;
using System.Collections;
using System.Collections.Generic;
```

```
[System.Serializable]
public class Step
{
    public int[] harvesterPosition;
    public int[] tractorPosition;
    public int[] containerPosition;
    public int containerLoad;
    public int[][] field;
}
```

```
[System.Serializable]
public class FarmData
{
    public Step step0;
    public Step step98;
    // Puedes añadir más propiedades según la estructura de tu JSON
}
```

```
public class MovimientoObjetos : MonoBehaviour
{
    public TextAsset farmDataFile; // Asigna el archivo JSON desde el editor de Unity
    private FarmData farmData;
    // Velocidad de movimiento del objeto
    public float velocidad = 5f;
    private bool ajusteRealizado = false;
    private bool recorridoAcabado = false;
    private bool recorridoAcabado2 = true;
    public float targetRotation = 180f; // Adjust this value as needed
    public float rotationSpeed = 180f;

    private Vector3 coordenadasObjetivoFinales = new Vector3(30.35f, 0.74f, -43.12f);

    IEnumerator RotateObjectRight()
    {
        float totalRotation = 0f;

        while (totalRotation < 180f)
        {
            // Calculate the rotation for each frame
            float rotationStep = Mathf.Min(rotationSpeed * Time.deltaTime, 180f - totalRotation);
```

```

// Rotate the object around the down (Y) axis
transform.Rotate(Vector3.down, rotationStep);

// Update total rotation
totalRotation += rotationStep;

yield return null; // Yield to the next frame
}

// Move the object left after rotation
Vector3 targetPosition = transform.position + new Vector3(8.3f, 0f, 0f);

while (transform.position.x < targetPosition.x)
{
    // Move the object left
    transform.Translate(Vector3.right * velocidad * Time.deltaTime);

    yield return null; // Yield to the next frame
}

// Optional: Perform any actions when rotation and movement are completed
Debug.Log("Rotation and movement completed!");
recorridoAcabado2 = false;
ajusteRealizado = true;
}

IEnumerator RotateObjectLeft()
{
    float totalRotation = 0f;

    while (totalRotation < 180f)
    {
        // Calculate the rotation for each frame
        float rotationStep = Mathf.Min(rotationSpeed * Time.deltaTime, 180f - totalRotation);

        // Rotate the object around the up (Y) axis instead of down
        transform.Rotate(Vector3.up, rotationStep);

        // Update total rotation
        totalRotation += rotationStep;

        yield return null; // Yield to the next frame
    }

    // Move the object right after rotation

```

```

Vector3 targetPosition = transform.position + new Vector3(-8.3f, 0f, 0f);

while (transform.position.x > targetPosition.x)
{
    // Move the object right instead of left
    transform.Translate(Vector3.left * velocidad * Time.deltaTime);

    yield return null; // Yield to the next frame
}

// Optional: Perform any actions when rotation and movement are completed
Debug.Log("Rotation and movement completed!");
recorridoAcabado = false;
ajusteRealizado = true;
}

void Start()
{
    if (farmDataFile != null)
    {
        // Obtén el contenido del archivo JSON como una cadena
        string json = farmDataFile.text;

        // Deserializar el JSON a un objeto de tipo FarmData
        farmData = JsonUtility.FromJson<FarmData>(json);

        // Ahora puedes acceder a la información como antes
        Debug.Log("Step 0 - Container Load: " + farmData.step0.containerLoad);
        Debug.Log("Step 1 - Harvester Position: " + farmData.step98.harvesterPosition[0] + ", "
+ farmData.step98.harvesterPosition[1]);
        // ... (puedes seguir accediendo a la información según la estructura de tu JSON)
    }
    else
    {
        Debug.LogError("El archivo farmData.json no se ha asignado.");
    }
    //StartCoroutine(RotateObjectLeft());
}

void Update()
{
    // Mueve el objeto hacia las coordenadas deseadas
    // MoverObjetoHaciaCoordenadas(farmData.step98.harvesterPosition[0], 0,
farmData.step98.harvesterPosition[1]);
    if (!recorridoAcabado)

```

```

    {
        MoverObjetoHaciaCoordenadasIda(0.47f, 0.74f,
-49);//-farmData.step98.harvesterPosition[1] - 40);
        //Debug.Log(farmData.step98.harvesterPosition[1]);
    }

    if (!recorridoAcabado2)
    {
        MoverObjetoHaciaCoordenadasRegreso(8.92f, 0.74f, 49);//
-farmData.step98.harvesterPosition[1] + 40);
        //Debug.Log(farmData.step98.harvesterPosition[1]);
    }

}

void MoverObjetoHaciaCoordenadasIda(float x, float y, int z)
{
    Vector3 coordenadasObjetivo = new Vector3(x, y, z);

    // Check if the adjustment has been made
    if (!ajusteRealizado)
    {
        coordenadasObjetivo += new Vector3(transform.position.x, transform.position.y,
transform.position.z);
        ajusteRealizado = true;
    }

    // Mover el objeto hacia las coordenadas objetivo
    transform.position = Vector3.MoveTowards(transform.position, coordenadasObjetivo,
velocidad * Time.deltaTime);

    // Verificar si el objeto ha llegado a las coordenadas objetivo
    if (transform.position == coordenadasObjetivo)
    {
        ajusteRealizado = false;
        recorridoAcabado = true;
        StartCoroutine(RotateObjectRight());
        Debug.Log("El objeto ha llegado a las coordenadas objetivo.");
    }
}

void MoverObjetoHaciaCoordenadasRegreso(float x, float y, int z)
{
    Vector3 coordenadasObjetivo = new Vector3(x, y, z);

    // Check if the adjustment has been made

```

```

    if (!ajusteRealizado)
    {
        coordenadasObjetivo += new Vector3(transform.position.x, transform.position.y,
transform.position.z);
        ajusteRealizado = true;
    }

    // Mover el objeto hacia las coordenadas objetivo
    transform.position = Vector3.MoveTowards(transform.position, coordenadasObjetivo,
velocidad * Time.deltaTime);

    // Verificar si el objeto ha llegado a las coordenadas objetivo
    if (transform.position == coordenadasObjetivo)
    {
        ajusteRealizado = false;
        recorridoAcabado2 = true;
        StartCoroutine(RotateObjectLeft());
        Debug.Log("El objeto ha llegado a las coordenadas objetivo.");
    }
}
}

```

4. URL del repositorio de GitHub y Presentación Final

Enlace al repositorio de GitHub en el que está toda la documentación y códigos generados*:

<https://github.com/Rodolfo-Charles/RetoJohnDeere.git>

Enlace a la presentación**:

https://drive.google.com/file/d/19rG1smsL1x3-tn_gPVTyurYSatQ1UaT7/view?usp=sharing

*Nota: Dres., fueron invitados a través de sus usuarios de GitHub (LuisAlbertoMunozUbando y rramirez-tec-mx).

**Nota: Dres., para ver la presentación, ingresen con su cuenta del Tec (amunoz@tec.mx y rramirez@tec.mx).

5. Documentación para instalación, configuración y ejecución de la simulación

Pasos para configurar Python y AgentPy en Visual Studio Code:

1. Descargar e instalar Visual Studio Code desde su [sitio web oficial](#).
2. Descargar e instalar Python desde [python.org](#).
3. Durante la instalación, asegurarse de marcar la opción “Add Python to PATH”.
4. Abrir Visual Studio Code.
5. Instalar la extensión de Python desde el marketplace de extensiones.
6. Abrir un archivo ‘.py’ o crear uno nuevo. Visual Studio Code te sugerirá seleccionar el intérprete de Python, elige el que instalaste.
7. Abrir la terminal integrada de Visual Studio Code.
8. Ejecutar ‘pip install agentpy’ en la terminal.

Otra opción si no se desea instalar un IDE (Integrated development environment) local es usar Google Colab.

Pasos para ejecutar código Python con AgentPy en Google Colab:

1. Abrir [Google Colab](#).
2. En una celda del notebook, escribir y ejecutar: ‘!pip install agentpy’.
3. Ejecutar las celdas con tu código para ver los resultados.

Pasos para instalar Unity con el último editor y Unity Hub:

1. Ir al [sitio web de Unity](#) y descargar Unity Hub.
2. Ejecutar el instalador descargado y seguir las instrucciones para instalar Unity Hub.
3. Abrir Unity Hub.
4. Ir a la pestaña “Installs” y hacer clic en “Add”.
5. Seleccionar la última versión de Unity.
6. Sigir las instrucciones para completar la instalación.

Pasos para clonar un repositorio de GitHub:

1. Descargar e instalar [Git](#).
2. Encontrar el repositorio en GitHub y copiar su URL.
3. Abrir una terminal o línea de comandos.
4. Navegar al directorio donde deseas clonar el repositorio.
5. Ejecutar ‘git clone [URL del repositorio]’.
6. Navegar al directorio clonado.

Pasos para ejecutar la simulación:

1. Ya que se instaló Unity con el último editor y Unity Hub y se clonó el repositorio (<https://github.com/Rodolfo-Charles/RetoJohnDeere.git>), abrir Unity Hub y seleccionar el botón de “Add” en “Projects”.
2. Seleccionar la carpeta de “SimulacionTractor” si se desea ver nuestra simulación 3D final de la solución al reto.
3. Seleccionar la carpeta de “pruebasProbuilder” si se desea ver nuestros modelos 3D hechos por nosotros desde cero.

6. Videos de instalación, configuración y ejecución de la simulación

Enlace a nuestro video en YouTube ejecutando nuestro código Python con AgentPy:

<https://youtu.be/JihB4sZ2ONE?si=7iE3zryyEFaD2SJT>

Enlace a nuestro video en YouTube para ver nuestra simulación 3D final de la solución al reto:

<https://youtu.be/cNP7qhQ1XkE?si=KPTVUvbqZzMKNQbA>

Videos de apoyo para instalación y configuración:

Video para configurar Python usando Visual Studio Code:

<https://youtu.be/NirAuEAblvo?si=l604GMbUyPF-vv3v>

Video para instalar Unity con el último editor (2022.3.12f1) y Unity Hub:

<https://youtu.be/dqJbCjPsqyg?si=C1jXEu7MV4zU1mji>

Video para clonar el repositorio (puede ser desde el min. 6:00):

https://youtu.be/i_23KUAEtUM?si=HK-f3cDFEs1rPh7W